Synopsis

Name of the College	**:**	Trinity College of Engineering and Research,Pune
Name of the Branch	**:**	Information Technology
Name of the Students	:	(IT-4010) Sanika Deshmukh
		(IT-4012) Trupti Dhandar
		(IT-4015) Vrushali Gawai
Project Group Number	:	PR 2025-26-05
Name of the Guide	:	Dr. Gajanan Arsalwad
Broad Area	:	Cloud Computing, Machine Learning
Project Title	:	SmartPlag – AI-Resistant Academic Plagiarism Checker with Code Analysis

1 Abstract

In the modern academic environment, the detection of plagiarism has become increasingly challenging due to the rise of AI-powered content generation tools and easy access to past student submissions. Traditional plagiarism detection systems such as Turnitin, Grammarly, and iThenticate are often expensive, lack integration with institutional archives, and primarily focus on text similarity without effectively handling paraphrased, AI-generated, or code-based plagiarism. Many educational institutions also lack a centralized platform for project submission, leading to inefficiencies and undetected internal plagiarism across batches.

SmartPlag addresses these limitations by providing a cost-effective, cloud-based platform designed specifically for academic institutions. It enables secure project submission and integrates advanced similarity detection for both text and source code using semantic analysis (TF-IDF/BERT) and code plagiarism detection tools (JPlag/MOSS). The system compares submissions against both online sources and an internal institutional database, ensuring comprehensive detection of copied or AI-restructured content. Role-based access for students,

faculty, and administrators streamlines the review process, while auto-generated PDF reports provide verifiable proof of plagiarism. By combining document and code analysis with institutional archiving, SmartPlag promotes academic integrity, reduces manual workload, and ensures originality in student work.

2 Introduction

In today's rapidly evolving academic landscape, maintaining originality and academic integrity has become increasingly challenging. With the growing accessibility of AI-powered tools like ChatGPT and widespread access to previous students' work, the submission of paraphrased or restructured projects has become common often bypassing traditional plagiarism detection tools. Most existing systems like Turnitin, Grammarly, or iThenticate are primarily designed for universities or publishers, are costly, and are often focused solely on text similarity without addressing internal institutional duplication or code-based plagiarism.

Moreover, most colleges do not have access to a centralized submission and archival system, which leads to difficulty in tracking project reuse across academic years. Internal plagiarism where students reuse previous batch submissions remains largely undetected due to the lack of such historical reference integration in current tools. Additionally, these platforms do not support simultaneous checking of project documentation and source code.

SmartPlag aims to bridge this gap by offering a cost-effective, cloud-based platform specifically built for college environments. The system will allow students to upload their project reports and source code, while faculty and administrators can review them through an integrated similarity checking engine. This engine will compare submissions against both online sources and the institution's historical database, allowing precise detection of copied content whether textual or logical.

By incorporating open-source plagiarism detection methods, intelligent text extraction, semantic comparison techniques, and internal project archives, SmartPlag not only automates the review process but also promotes academic honesty and originality. The system also ensures secure role-based access for students,faculty,and admin, making it a complete academic submission and verification platform tailored to the modern classroom.

3 Problem Statement

Most educational institutions lack a unified and affordable platform for secure project submission and plagiarism detection. Existing tools are costly, do not support internal student-to-student comparison, and are not integrated with cloud-based academic workflows. This leads to inefficiency, manual workload, and risks of academic dishonesty.

Current plagiarism detection tools:

•Are paid, not accessible to all colleges

•Do not compare internal student submissions

•Don't support code + document checking in one system

•Lack student–teacher workflow and college-based archival verification

Our project fills this gap using open-source cloud technology to build a smart, scalable system for submission, comparison, and verification.

4 Literature Review

AI-generated source code detection is a relatively recent research area, emerging alongside the rapid growth of LLM coding capabilities (OpenAI, 2022; Anthropic, 2024; DeepSeek, 2025). Early approaches such as Whodunit (Idialu et al., 2024) leveraged surface-level statistical and stylometric features extracted from code and its Abstract Syntax Tree (AST). While effective against earlier models, these methods struggle to detect outputs from modern, high-performing LLMs capable of mimicking human coding styles.

To improve detection robustness, researchers have proposed more advanced methods across four paradigms: zero-shot detectors (e.g., DetectGPT, DetectCodeGPT, AIGC Detector) that rely on token-level statistics; embedding-based methods (e.g., CodeT5+, CodeX-Embed) that transform code into high-level semantic representations for classification; fine-tuning-based approaches (e.g., GPTSniffer, RoBERTa) trained on labeled code datasets; and pretrained LLM + downstream detectors (e.g., Raidar, BiScope) that extract semantic or

statistical features for secondary classification.

Several datasets and benchmarks have emerged to evaluate such methods, including Suh et al. (2025) with a multi-language function-level dataset, Pan et al. (2024) and CoDet-M4 (Orel et al., 2025) incorporating adversarial perturbations, and LLMGCode (Xu et al., 2024) expanding language diversity. Nonetheless, these benchmarks exhibit key shortcomings: most cover only a small subset of programming languages, rely primarily on open-source or less capable LLMs, and lack integration of latest reasoning models. Furthermore, adversarial evaluation—especially source-code-specific paraphrasing—is rare, limiting insights into detector robustness under real-world evasion tactics.

The CodeMirage benchmark addresses these limitations by offering a large-scale, multilingual dataset spanning 10 programming languages, containing both original and paraphrased code generated by 10 state-of-the-art production-level LLMs (including reasoning-capable models). It systematically evaluates detectors across all four methodological paradigms under multiple configurations—in-distribution, paraphrase, cross-model, and cross-model paraphrase—using complementary metrics (F1, TPR@FPR=10

Table 1: Literature Review of 2025 Papers on AI-Generated Code and Plagiarism Detection

Name	**Author / Year**	**Methodology**	**Limitations**	**Research Gap**
CodeMirage: A Multi-Lingual Benchmark for Detecting AI-Generated and Paraphrased Source Code	Guo et al., 2025	Large-scale benchmark (10 languages, 210k samples) with human, AI-generated, and paraphrased code; evaluates detectors across four paradigms and configurations	Benchmark only; no novel detector; excludes real-world deployment integration	Build detection models leveraging CodeMirage for deployment-ready plagiarism/security applications
Evaluating Software Plagiarism Detection in the Age of AI	Sağlam & Schmid, 2025	Tests traditional tools like JPlag against AI-assisted obfuscation	Focused on education; no new detection algorithms	AI-resilient plagiarism detection for semantic-preserving transformations in multiple languages
Domain Gating Ensemble Networks (DoGEN)	2025	Multi-expert ensemble with domain gating for out-of-domain text detection	Focused on text; no code or multilingual programming evaluation	Adapt for cross-domain, multilingual AI-generated code detection
The Failure of Plagiarism Detection in Competitive Programming	Dickey, 2025	Case study on MOSS limitations with AI-generated/obfuscated code	Context-specific; lacks generalized detection framework	Develop generalized AI-aware plagiarism detection adaptable to multiple coding contexts
AI on Academic Integrity and Plagiarism Detection	Leong Zhang, 2025	Survey of AI's role, ethics, and privacy in plagiarism detection for text/code	Broad and conceptual; lacks empirical datasets	Empirical validation of AI-integrated plagiarism detection in code contexts
Reassessing Academic Integrity in the Age of AI	ScienceDirect, 2025	Systematic review of AI's impact on academic integrity	High-level; no technical/code-specific details	Domain-specific code plagiarism detection frameworks
Enhancing Plagiarism Detection in Marathi	Mutsaddi Choudhary, 2025	TF-IDF + BERT weighted ensemble for Marathi text plagiarism	Language-specific; not adapted for code	Extend multilingual NLP ensembles to code plagiarism
A Comprehensive Strategy for Identifying Plagiarism in Academic Submissions	Umm Al-Qura Univ., 2025	Strategic framework combining technical and procedural detection measures	Lacks algorithmic depth; no AI-code focus	Integrate AI-resistant code plagiarism detection into academic systems

5 Significance

The proposed system, *SmartPlag*, significantly strengthens academic integrity by introducing an advanced, institution-specific plagiarism detection framework. It ensures fairness and originality by:

- Cross-checking student submissions against **previous batches' reports and code**, stored in a secure internal repository.
- Performing **multi-layered similarity analysis** combining semantic text similarity (TF-IDF/BERT) with code similarity detection (JPlag/MOSS).
- Automatically generating **comprehensive PDF plagiarism reports** containing matched batch details, group IDs, similarity percentages, and highlighted plagiarism evidence.
- Providing an **interactive faculty dashboard** to review matched results, add comments, and make informed accept/reject decisions efficiently.
- Maintaining a **centralized, secure, and role-based system** ensuring data privacy, fast retrieval, and transparent decision-making.

By leveraging historical project data, SmartPlag moves beyond internet-only plagiarism checks and enables **deep internal plagiarism detection**. This dual-layer document and code verification ensures more accurate results and offers a streamlined workflow for faculty to uphold academic integrity at scale.

6 Proposed Work

6.1 Objectives

1. Develop a secure **cloud-based project submission portal** for both students and faculty.
2. Integrate **automatic plagiarism detection** for both textual content and source code.
3. Connect the system with **historical internal project databases** to detect cross-batch plagiarism.

4. Automatically **generate detailed PDF plagiarism reports** summarizing similarities and matched project details.
5. Implement **role-based workflows** for students, teachers, and administrators to ensure secure access control.
6. Provide **faculty decision tools** to review results, leave remarks, and approve or reject submissions efficiently.

6.2 Methodology

1. **Student Upload** – Students submit their project reports and/or source code through the portal.
2. **Data Extraction** – The system extracts the project title, abstract, problem statement, and relevant content.
3. **Similarity Analysis**:
   - **Text:** Compared using semantic similarity methods like TF-IDF, cosine similarity, and BERT embeddings.
   - **Code:** Compared using structural and token-based detectors such as JPlag and MOSS.
4. **Internal Cross-Check** – Matches are searched in the college's internal database containing past student submissions.
5. **Faculty Review Dashboard** – Displays matched projects with **group ID, submission year, and similarity evidence**.
6. **Decision Logging** – Faculty record the accept/reject decision along with optional comments.
7. **PDF Report Generation** – A finalized plagiarism summary report is generated using **jsPDF**, containing match details, similarity scores, and proof highlights.

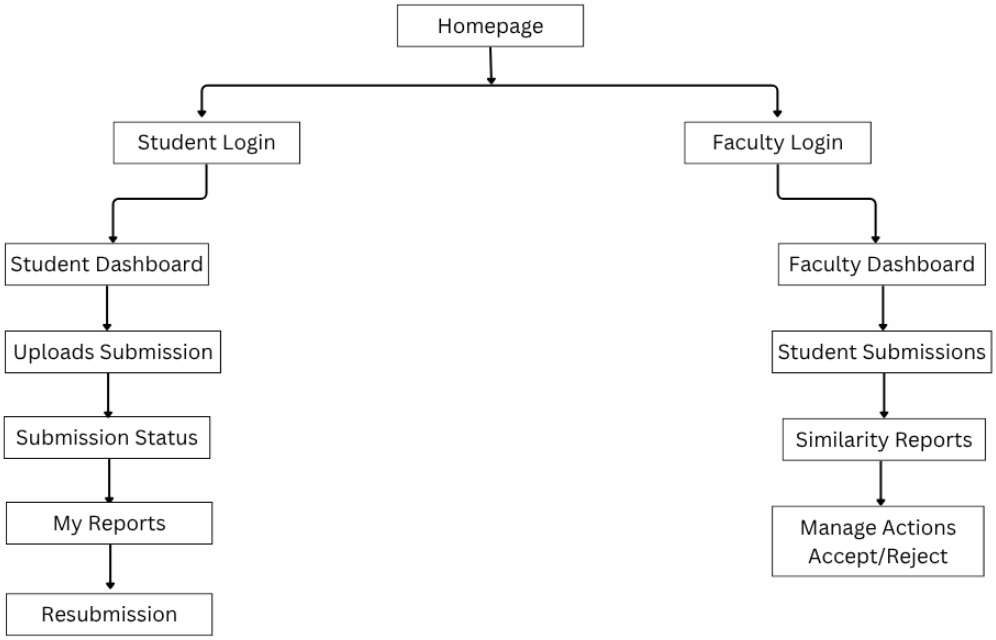

Figure 1: Interface Flowcharts

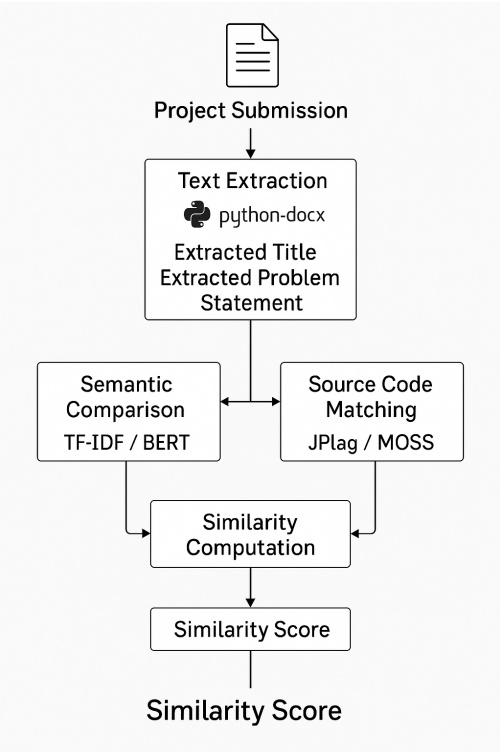

Figure 2: WorkFlow

6.3 Schedule of Project Stage-I Work

Week No	Activity Planned
01	Finalization of Project Domain, Submission of 4–5 Proposed Topics, and Literature Survey (minimum 2–3 papers per topic).
02	Identification of Problem Statement/Title based on literature review, feasibility analysis, and gap identification. **Pre-Review Presentation** and Topic Finalization.
03	Documentation of Motivation, Objectives, and Defined Scope of the Finalized Project.
04	Comparative Study of Existing Methods/Tools, Feasibility Assessment, and Listing of Required Resources (tools, technologies, hardware/software).
05	Overview of Proposed System, Expected Output, and Basic Design. **Project Review – 1**.
06	System Architecture and Initial Design Elements (UML,Context-level DFD, High-Level Block Diagrams).
07	Documentation of System Requirements – Functional and Non-Functional.
08	Preparation of the SRS (Software Requirements Specification) Document as per the finalized problem statement.
09	Requirement Analysis using UML/ER Diagrams, Use Case Diagrams, Class/Activity Diagrams. **Project Review – 2**.
10	Detailed System Architecture, Design Specifications, and Algorithmic Analysis.
11	Module-level Testing Plan and Initial Test Case Design.
12	Detailed Design – Expanded DFD Levels, Module Specifications, and Data Dictionary
13	Development of Initial Working Modules (at least 3–4). Submit partial coding with documentation. Continue survey paper preparation.
14	Drafting of Research Paper, Finalization of Project Report Format, Timeline Planning. **Project Review – 3**.

Table 2: Schedule of Project Stage-I Work

Apart from above schedule, some additional weeks are required for below mentioned activities:

- Conference/Journal paper Preparation and Presentation.
- Semester-I project stage I Report Preparation.
- Semester-II project stage II Final project presentation and Report Preparation.

6.4 Software Requirements:

Software Requirements:

- **Platforms:** Web-based application (Cross-platform – accessible via browser)
- **Frontend Technologies:** React.js, Tailwind CSS, jsPDF (for report generation)
- **Backend Technologies :**Node.js, Express.js, Multer (file handling), JWT/Firebase Auth
- **Languages Used :** JavaScript (Frontend Backend), Python
- **Database :** MongoDB Atlas
- **Cloud Storage Services:** Firebase Storage or AWS S3 (for project file storage), Vercel/Render for deployment
- **Plagiarism Detection Tools:**JPlag / MOSS (for code), TF-IDF or BERT (for semantic text similarity)

References

[1] Evaluating Software Plagiarism Detection in the Age of AI: Automated Obfuscation and Lessons for Academic Integrity – Sağlam Schmid (May 2025). Analyzes automated obfuscation techniques and shows how traditional detectors (e.g. JPlag) can be defeated by AI-assisted transformations

[2] Domain Gating Ensemble Networks for AI-Generated Text Detection – (2025) A framework called DoGEN (Domain Gating Ensemble Networks) that enables robust detection of AI-generated content across unseen domains. Although primarily focused on text, its multi-expert ensembling approach can be adapted for code stylometry across different programming styles or domains. Demonstrates state-of-the-art performance on out-of-domain detection tasks, offering insights for cross-domain code detection systems.

[3] CodeMirage: A Multi-Lingual Benchmark for Detecting AI-Generated and Paraphrased Source Code from Production-Level LLMs – Guo et al. (May 2025). Introduces a benchmark across 10 languages and evaluates robustness of detection models under realistic AI paraphrasing attacks

[4] The Failure of Plagiarism Detection in Competitive Programming – Dickey (May 2025). Examines limitations of tools like MOSS in programming course settings, especially with AI-generated or obfuscated code

[5] AI on Academic Integrity and Plagiarism Detection – Leong Zhang (2025). Surveys AI's role and ethical/privacy implications in plagiarism detection systems, addressing non-textual forms including code plagiarism

[6] Reassessing Academic Integrity in the Age of AI: A Systematic Review – ScienceDirect (2025). Highlights research gaps in AI's impact on academic integrity, including code-based plagiarism

[7] Artificial Intelligence and Plagiarism: Challenges, Detection, and Prevention – IJERD (Oct 2024). Though published in late 2024, its implications extend into 2025 as a foundation for interdisciplinary integrity frameworks

[8] Discovering and exploring cases of educational source code plagiarism with Dolos – Maertens et al. (Feb 2024). Presents Dolos, a full-stack ecosystem, with clustering dashboards and similarity detection for educational code plagiarism

[9] An Empirical Study on Automatically Detecting AI-Generated Source Code: How Far Are We? – Suh et al. (Nov 2024). Proposes ML-based detection using AST embeddings and fine-tuned LLMs, outperforming existing detectors with F 82.6

[10] Learning AI Coding Style for Software Plagiarism Detection – Ambati et al. (Oct 2024). Investigates AI-specific coding style patterns to distinguish AI-generated code via authorship attribution methods

[11] A Support Vector Machine Approach for Plagiarism Detection in Python Code Submissions – Gandhi, Gopalan Prasad (2024). Uses textual and syntactic Python-specific features with SVM to detect plagiarism in educational assignments

[12] Automated Detection of AI-Obfuscated Plagiarism in Modeling Assignments – Sağlam et al. (2024). Focuses on identifying AI-based obfuscation in code/modeling assignments using structural detection techniques

[13] Automated Detection of AI-Obfuscated Plagiarism in Modeling Assignments – Sağlam et al. (2024). Focuses on identifying AI-based obfuscation in code/modeling assignments using structural detection techniques

[14] Survey on Plagiarism Detection in LLMs and Generated Code – Pudasaini et al. (2024). A broad review of detection approaches, evasion strategies, datasets, and policy implications around code and text plagiarism (similar in scope to Leong Zhang)

[15] Source Code Anomaly Detection for AI-Assisted Submissions – (2024). A study showing high precision in detecting anomalies in AI-assisted code submissions using semantic analysis techniques

[16] Whodunit: Classifying Code as Human Authored or GPT-4 Generated – (2024) A study achieving 91% F1-score in detecting GPT-4 generated code submissions using stylometric

and structural features like AST depth, token patterns, and code layout. Demonstrates high reliability without relying on superficial cues.

[17] Sağlam and Schmid (May 2025) : Evaluating Software Plagiarism Detection in the Age of AI: Automated Obfuscation and Lessons for Academic Integrity

[18] Guo et al. (May 2025) : CodeMirage: A Multi-Lingual Benchmark for Detecting AI-Generated and Paraphrased Source Code

[19] Dickey (May 2025) : The Failure of Plagiarism Detection in Competitive Programming

[20] Mehsen Joshi (Apr 2024) : Detection of Source Code Plagiarism Utilizing an Approach Based on Machine Learning

[21] Mutsaddi Choudhary (Jan 2025) : Enhancing Plagiarism Detection in Marathi ... Weighted Ensemble of TF-IDF and BERT

[22] Umm Al-Qura University Journal (2025) : A Comprehensive Strategy for Identifying Plagiarism in Academic Submissions

Sanika Deshmukh
Trupti Dhandar
Vrushali Gawai
Name of The Students

Dr.Gajanan Arsalwad
Name of the Guide

Dr. V. S. Gaikwad
Head of the Department(IT)